

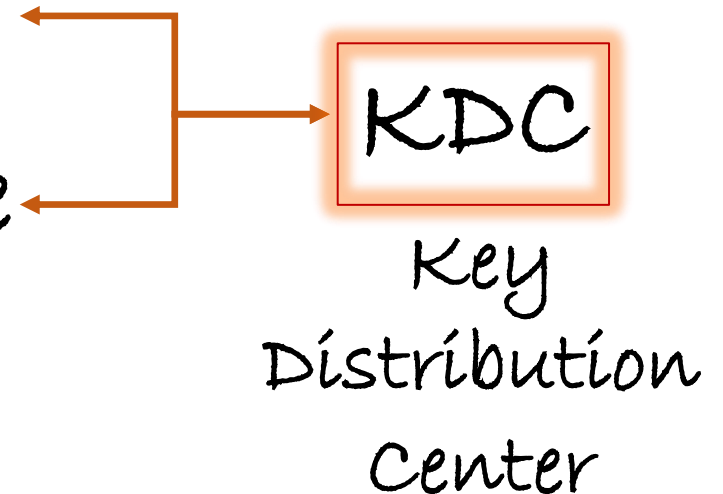
Clase 5

Críptografía de Clave Privada



LIMITACIONES

1. Distribución de Clave
2. Almacenamiento de clave
3. Sistemas Abiertos



KDC

Key Distribution Center

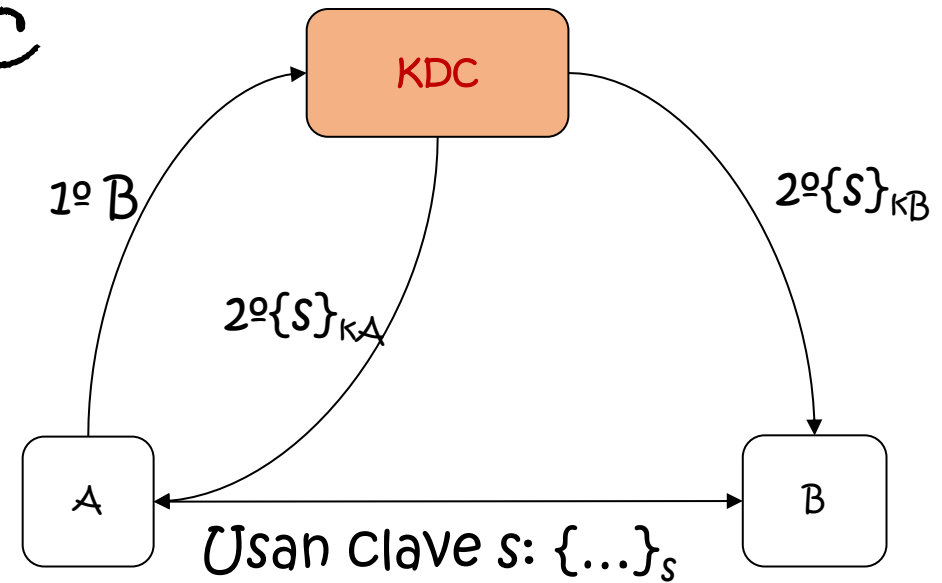
Todos comparten k_u con KDC
(k_A sí es A, k_B sí es B, etc.)

1º: $A \rightarrow KDC: B$

2º: $KDC \rightarrow A: \{S\}_{k_A}$

$KDC \rightarrow B: \{S\}_{k_B}$

Luego A se comunica con B usando s.



Ej: Needham Schroeder

KDC



- cada uno guarda sólo k_u
- Sólo un sitio de riesgo: el KDC



Centralización:

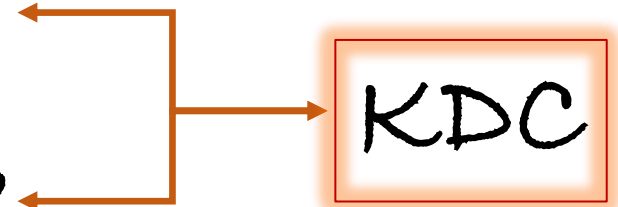
- Ataque al KDC
- Falla en KDC

Críptografía de Clave Privada



LIMITACIONES

1. Distribución de Clave
2. Almacenamiento de clave
3. Sistemas Abiertos (Open Systems)



Public Key Revolution



Public Key Revolution

Diffie - Hellman



Funciones fáciles en un sentido, difíciles de invertir

Dos claves: { p/encrípcion (emisor) pública

p/desencrípcion (recep.) privada

ASIMÉTRICOS

Public Key Revolution

Díffie y Hellman proponen 3 primitivas:

1ª) Encriptación

→ RSA/El Gamal

2ª) MAC (firma digital)

→ permite no repudio.

3ª) intercambio de claves

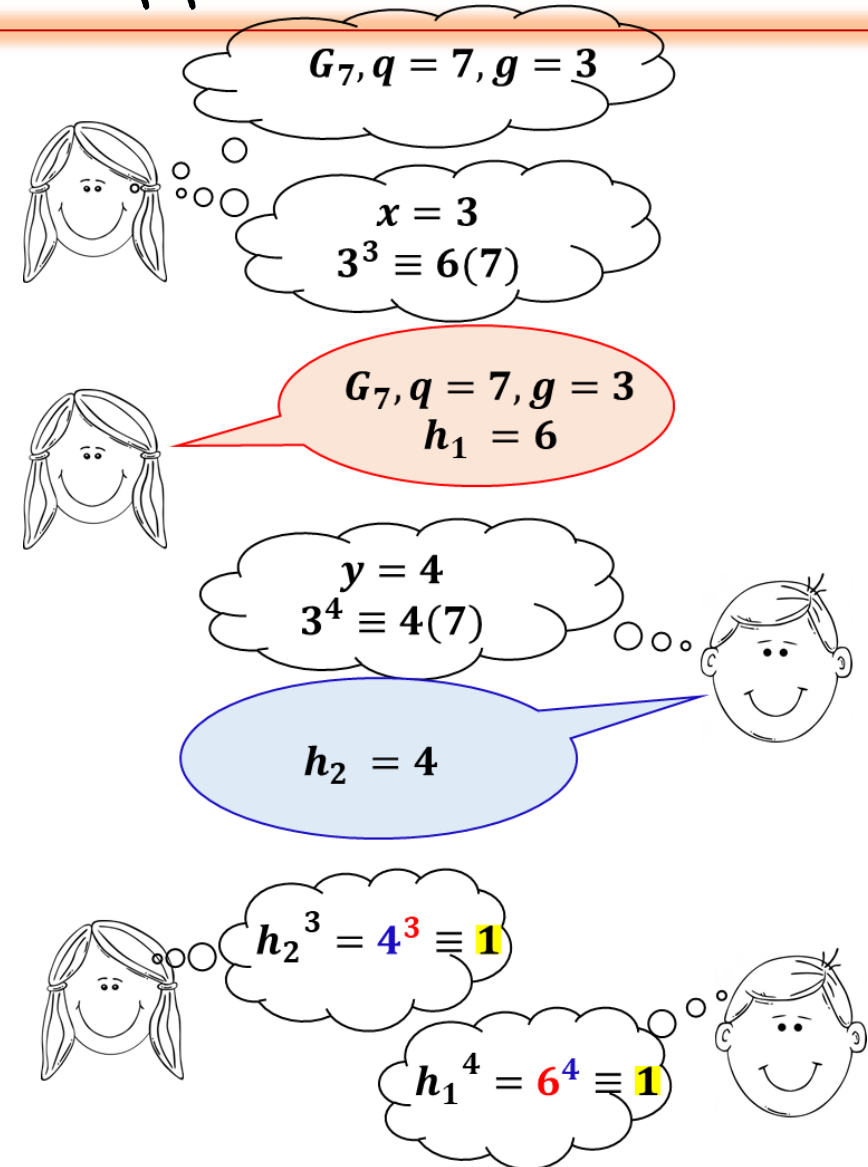
→ presentan este protocolo completo.

Intercambio de claves de Diffie-Hellman

1. A: (G, q, g)
2. A: elige $x \leftarrow \mathbb{Z}_q$ aleatoriamente y uniforme y calcula $h_1 = g^x$
3. A: envía (G, q, g, h_1) a B.
4. B: elige $y \leftarrow \mathbb{Z}_q$, calcula $h_2 = g^y$, lo envía a A y calcula $k_B = h_1^y$
5. A: calcula $k_A = h_2^x$

$$k_A = k_B \\ = g^{xy}$$

En $\mathbb{Z}_7$ $g = 3$: $3^0 = 1$; $3^1 = 3$; $3^2 = 2$;
 $3^3 = 6$; $3^4 = 4$; $3^5 = 5$





Ataques

- Impersonating / Masquerading: uno solo de los participantes ejecuta el protocolo y el adversario se hace pasar por el otro participante.
- MIM Man in the Middle: las dos partes ejecutan el protocolo. El adversario intercepta y modifica mensajes que se envían.
- Replay
- Key Reuse

Esquemas ASIMÉTRICOS

Gen $\rightarrow$ 2 claves:

- p /encrípcion (p_k) pública
- p /desencrípcion (s_k) privada

La seguridad subyace en mantener s_k secreta y no p_k



- Resuelve distribución de claves
- Sólo se guarda una s_k para comunicarme con varios.
- s_k y p_k se generan por antícipado.



- Más lento.

GenRSA: algoritmo de generación de claves

$(N, p, q) \leftarrow \text{GenModulus}(1^n)$

con: $\Phi(N) = (p-1)(q-1)$

Elegir e : $\text{mcd}(e, \Phi(N)) = 1$

Calcular d : $[e^{-1} \bmod (\Phi(N))]$

$\langle N, e \rangle$ es p_k $\langle N, d \rangle$ es s_k

GenRSA con openssl

```
$ openssl genrsa -out privada.pem 2048
```

```
Generating RSA private key, 2048 bit long modulus
```

```
.....+++  
.....+++
```

```
e is 65537 (0x10001)
```

```
$ ls -l
```

```
-rw-r--r-- 1 siles staff 1679 Sep  1 00:00 privada.pem
```

```
$ file privada.pem
```

```
privada.pem: PEM RSA private key
```

Cuanto más grande la
clave, más tarda en
calcular los primos p y q

Clave privada en openssl

El siguiente comando permite obtener una versión en formato texto de la clave privada RSA generada, con todos los parámetros asociados:

```
$ openssl rsa -in privada.pem -text -out privada.txt  
writing RSA key
```

En privada.txt se va a encontrar los números p y q , el módulo N , los valores e y d y otros valores para agilizar los cálculos.

Más información sobre estos temas:

<https://www.flu-project.com/2019/10/generacion-claves-rsa-Parte-3.html>

Esquema de Cifrado de Clave Pública

1. Gen: $(p_k, s_k) \leftarrow \text{Gen}(1^n)$
2. Enc: $c \leftarrow \text{Enc } p_k(m)$
3. Dec: $m := \text{Dec } s_k(c)$



Sí el cifrado público es determinístico
NO es CPA - Secure

Esquema RSA encriptación

1. Gen: ejecuta **GenRSA** $p_k \langle N, e \rangle, s_k \langle N, d \rangle$
2. Enc: $c := [m^e \bmod N]$
3. Dec: $m := [c^d \bmod N]$

$$c^d \bmod N = c^{e^{-1} \bmod (\varphi(n))} \bmod N = (m^e)^{e^{-1} \bmod (\varphi(n))} \bmod N = m^1 = m$$

Esquema de Cifrado Híbrido

Público-asím (π) + privado-sím (π')

1. Gen^{hy} : $(p_K, s_K) \leftarrow \text{Gen}(1^n)$
2. Enc^{hy} : Elegir $k \leftarrow \{0,1\}^n$.
Calcular $c_1 \leftarrow \text{Enc } p_K(k)$ cifrado asimétrico
y $c_2 \leftarrow \text{Enc}'_k(m)$ cifrado simétrico
y emitir (c_1, c_2)
3. Dec^{hy} : $k := \text{Dec } s_K(c_1)$
 $m := \text{Dec}'_k(c_2)$

CPA seguro siempre que:
 π CPA seguro
 π' Seguro ante Eavesdropping

Firma Dígital vs MAC

1. Gen: $(p_k, s_k) \leftarrow \text{Gen}(1^n)$
2. Sign: $\sigma \leftarrow \text{sign } s_k(m)$
3. Vrfy: $b := \text{Vrfy } p_k(m, \sigma)$

Esquema MAC (Gen, Mac, Vrfy)

- Gen: $k \leftarrow \text{Gen}(1^n)$
- Mac: $t \leftarrow \text{Mac}_k(m)$
- Vrfy: $b := \text{Vrfy}_k(m, t)$

emisor envía $\langle m, t \rangle$
 $b = 1$ si es válido

- Ambos aseguran integridad.
- Fácil distribución de clave
- Una sola firma puede ser verificada por cualquier receptor
- Verificables públicamente
- Provee "no repudio"
- Una clave p/c/receptor
- Una MAC p/c/clave
- No verificables públicamente
- No Provee "no repudio"
- Más eficiente

Textbook RSA para firma

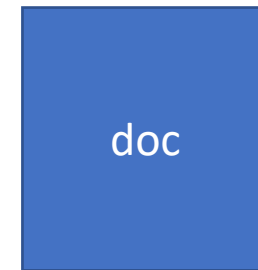
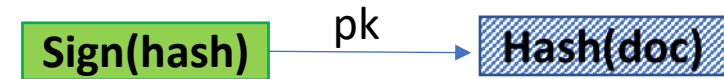
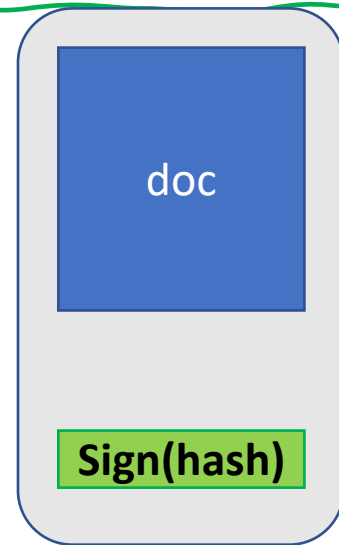
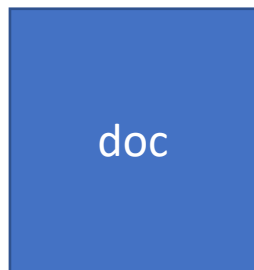
1. Gen: ejecuta ~~GenRSA~~ $p_k \langle N, e \rangle, s_k \langle N, d \rangle$
2. Sign: $\sigma := [m^d \bmod N]$
3. Vrfy: $m := [\sigma^e \bmod N]$



INSEGURO
(Ataque de no mensaje)

Hash RSA reduce riesgo

1. Gen: $(p_k, s_k) \leftarrow \text{Gen}(1^n)$
2. Sign: $\sigma := [(H(m))^d \bmod N]$



= ≠



PKI: Public Key Infrastructure

Define cómo distribuir $\langle p_k, s_k \rangle$

Certificados

- Jerarquías
 - Validez
- Expiración
Revocación

TLS= Transport Layer Security

Proporciona cifrado, autenticación e integridad entre ambos extremos de la conexión

Confidencialidad: Combina cifrado simétrico y asimétrico

Autenticación: Auténtica cliente y autentica servidor

Integridad: Usa hash

- Applied Cryptography - Bruce Schneier
- TLS = RFC 2246 (primera versión) y RFC 8446 (TLS 1.3)